



Duale Hochschule Sachsen

- Studienrichtung Medieninformatik -

Pflichtenheft Softwaretechnik

Lord of Nothing

Gruppenmitglieder:

Justin Hesse

Luca Kaden

Valentin Maria Marcinek

Christopher Aaron Sabbach

Inhaltsverzeichnis

Inhaltsverzeichnis	II
1 Zielbestimmung	1
1.1 Musskriterien	1
1.2 Wunschkriterien	2
1.3 (Abgrenzungskriterien)	3
2 Produkteinsatz	4
2.1 Anwendungsbereiche	4
2.2 Zielgruppen	4
2.3 Betriebsbedingungen	4
3 Produktübersicht	5
4 Funktionale Anforderungen	6
5 Nicht-Funktionale Anforderungen	7
6 Produktdaten	8
6.1 Datensätze und Datenobjekte	8
7 Laufzeitumgebung [zum Betrieb der Software]	9
7.1 Software	9
7.2 Hardware	9
7.3 (Orgware)	9
8 Entwicklungsumgebung [zur Entwicklung der Software]	10
8.1 Software	10
8.2 Hardware	10
8.3 (Orgware)	10
9 Entwicklungshistorie	11
9.1 1. Sprint: 19.03.2026 - 30.03.2026	11
9.2 2. Sprint: 30.03.2026 - 09.04.2026	11
9.3 3. Sprint: 09.04.2026 - 16.04.2026	12
9.4 4. Sprint: 16.04.2026 - 23.04.2026	12
9.5 5. Sprint: 23.04.2026 - 05.05.2026	12
9.6 6. Sprint: 05.05.2026 - 11.05.2026	13
9.7 7. Sprint: 11.05.2026 - 17.05.2026	13

1 Zielbestimmung

Für die Festlegung von Zielen im Projekt wurde für das Brainstorming eine Tabelle angelegt. Nachdem alle Ideen aufgeführt worden waren, wurden diese durch gemeinsame Absprache in eine von sechs Prioritätsstufen eingeordnet: Prototyp (erste Anwendung, die Grundkonzepte und grundlegende Funktionalitäten beinhaltet), MVP (minimum viable product; bildet gemeinsam mit den Prototyp-Funktionen das Mindestmaß der Funktionen des Endprodukts), MLP-Stufen 1 bis 3 (most lovable product; beinhaltet Funktionen, welche die Spielerfahrung und das Spiel selbst verbessern würden), sowie Out of Scope (alle Funktionen, welche nicht umgesetzt werden).

1.1 Musskriterien

Die Musskriterien beinhalten alle Funktionen der Prioritätsstufen „Prototyp“ sowie „MVP“.

Nr.	Prioritätsstufe	Feature	Beschreibung	Status
1	Prototyp	Gridmap	Es gibt ein Spielfeld mit einer Gitterstruktur	Umgesetzt
2	Prototyp	Gebäude platzieren	Es können Gebäude auf der Gridmap platziert werden	Umgesetzt
3	Prototyp	Build Menu	Es gibt ein Menü zum Auswählen und Bauen von Gebäuden	Umgesetzt
4	MVP	Ressourcen	Es werden Ressourcen generiert (z.B. Holz, Stein, Nahrung)	Umgesetzt
5	MVP	versch. Gebäudetypen	Es gibt verschiedene Gebäudetypen	Umgesetzt
6	MVP	Bewohner	Es werden Bewohner generiert, die den Gebäuden zugewiesen werden können	Umgesetzt
7	MVP	Game Over-Mechanic	Es kann zum Ende eines Runs kommen, wenn bestimmte Voraussetzungen erfüllt sind	Umgesetzt
8	MVP	Menü	Es gibt ein Hauptmenü und Untermenü für verschiedene Funktionen wie beispielsweise Einstellungen	Umgesetzt
9	MVP	Raids	Es werden Raids generiert, welche dein Dorf angreifen; Diese werden mit der Zeit schwerer, da mehr Raider generiert werden	Umgesetzt

1.2 Wunschkriterien

Die Wunschkriterien beinhalten alle Funktionen der Prioritätsstufen „MLP-Stufe 1“, „MLP-Stufe 2“ und „MLP-Stufe 3“. Dabei besitzt Stufe 1 die höchste, Stufe 3 die niedrigste Priorität.

Nr.	Prioritätsstufe	Feature	Beschreibung	Status
10	MLP-Stufe 1	Spielfeld mit Biomen	Es gibt verschiedene Biome wie Gebirge oder auch Flüsse, welche andere Voraussetzungen, Vorteile und Nachteile bieten	Nicht umgesetzt
11	MLP-Stufe 1	Introduction/Tutorial	Es gibt eine kleine Spielanleitung, die dem Spieler den Start vereinfacht	Umgesetzt
12	MLP-Stufe 1	weitere random Events	Es gibt weitere random Events wie Dürre, Krankheiten, schlechte Ernte, harter Winter, Dorffeuern,...	Nicht umgesetzt
13	MLP-Stufe 1	Hintergrundmusik	Es gibt Hintergrundmusik, die das Spiel immersiver macht	Umgesetzt
14	MLP-Stufe 1	Soundeffekte	Es gibt Soundeffekte für verschiedene Buttons/Events	Umgesetzt
15	MLP-Stufe 1	Grafik/Kamera	2D-Spiel, Kamera mit Topdown-View; Simulierte 3/4-Perspektive bzw. Dreipunktperspektive; kleine Animationen; Retro-Pixel-Look, 2D-Menügrafiken	Umgesetzt
16	MLP-Stufe 1	Spielstand speichern	Es kann ein Spielstand gespeichert und bei erneutem Starten des Spieles wieder geladen werden	Umgesetzt
17	MLP-Stufe 2	Hindernisse auf dem Spielfeld	Es gibt Hindernisse wie Bäume oder Felsen, welche vor dem Bebauen des Spielfeldes entfernt werden müssen	Nicht umgesetzt
18	MLP-Stufe 2	Run-Historie	Es gibt eine Übersicht über vergangene und Gespeicherte Runs	Nicht umgesetzt

19	MLP-Stufe 2	Forschung	Neue Gebäude; Gebäude-Upgrades; neue Truppen; Skilltree	Nicht umgesetzt
20	MLP-Stufe 3	Zufällige Spielfeldgenerierung	Das Spielfeld wird zufällig generiert	
21	MLP-Stufe 3	Schwierigkeitsstufen	Es gibt Schwierigkeitsstufen (Easy/Hard/Nothingness), wodurch das Spiel anspruchsvoller wird	Nicht Umgesetzt

1.3 (Abgrenzungskriterien)

Die Abgrenzungskriterien beinhalten alle Funktionen der Prioritätsstufe „Out of Scope“ sowie weitere.

Nr.	Feature	Begründung
22	Stimmungsbarometer	Durch Änderungen am Spielkonzept obsolet
23	Individuelle Bewohner mit Skills	Übersteigt den Umfang des Projekts
24	Multiplayer	Übersteigt den Umfang des Projekts
25	Mobile Version	Übersteigt den Umfang des Projekts

2 Produkteinsatz

2.1 Anwendungsbereiche

„Lord of Nothing“ ist ein roguelike Aufbau-Strategiespiel für den Desktop-PC (Windows/MacOS). Es wird im Bereich der digitalen Freizeitunterhaltung eingesetzt und ist für den Einzelspielerbetrieb ausgelegt. Der Spieler agiert als Herrscher eines mittelalterlichen Dorfes, welches er aufbaut, verwaltet Ressourcen und Bewohner und verteidigt das Dorf gegen Angriffe (Raids) von Banditen.

2.2 Zielgruppen

Das Spiel richtet sich an Gelegenheitsspieler ab 12 Jahren mit grundlegenden Kenntnissen im Umgang mit einem PC. Vorkenntnisse im Strategiespiel-Genre sind von Vorteil, aber nicht erforderlich, da ein kleines textbasiertes Tutorial zu Beginn des Spiels in die grundlegenden Mechaniken einführt und Tipps gibt.

2.3 Betriebsbedingungen

Das Spiel ist für den Offline-Betrieb ausgelegt und erfordert keine Internetverbindung. Es ist nicht für den Dauerbetrieb vorgesehen. Es gibt keine weiteren besonderen Betriebsbedingungen.

3 Produktübersicht

Die folgende Abbildung gibt einen groben Überblick über die Anwendungsfälle des Spiels und deren Akteure. Eine detaillierte Beschreibung der einzelnen Funktionen folgt in Abschnitt 4.

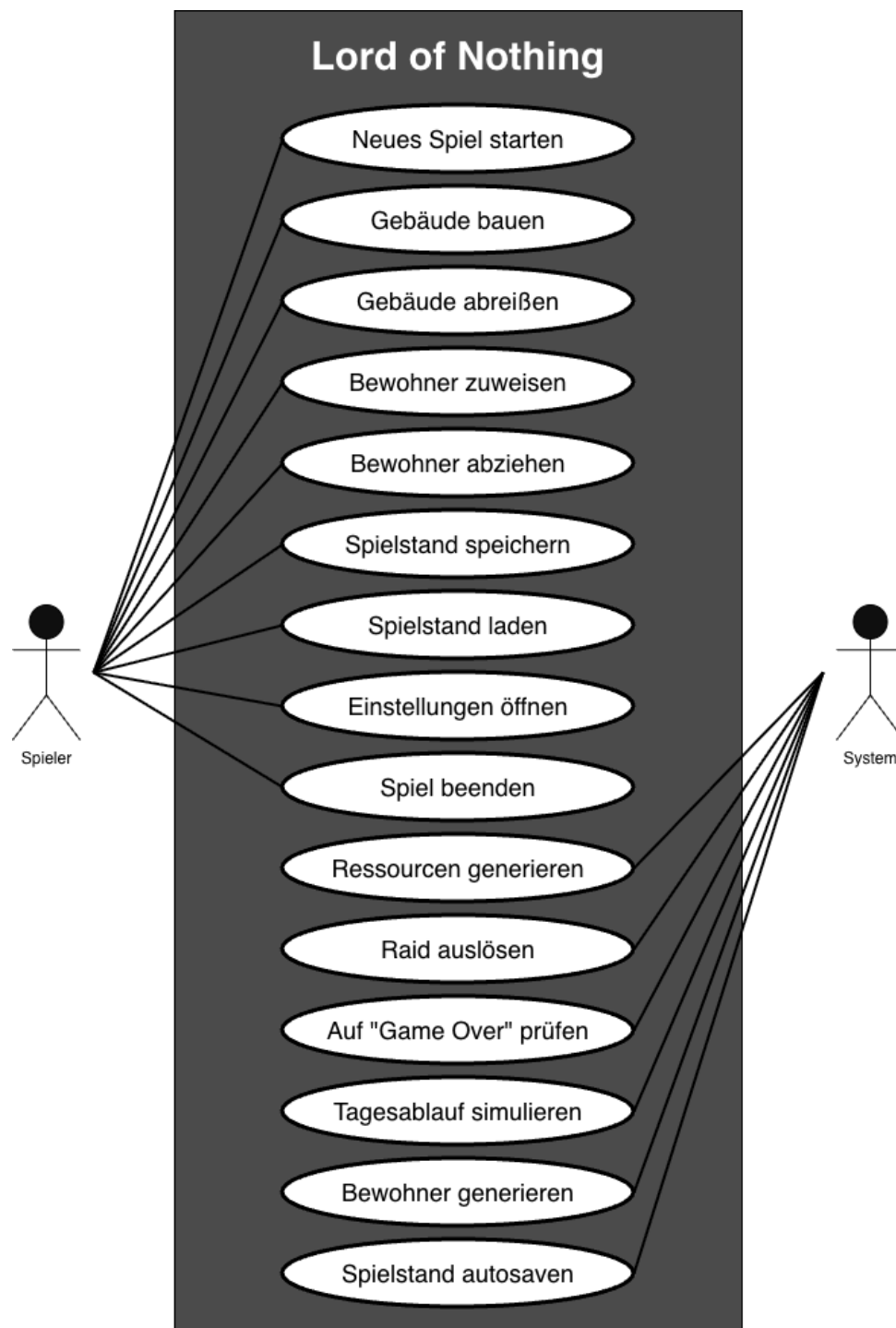


Abbildung 1: Anwendungsfalldiagramm „Lord of Nothing“

4 Funktionale Anforderungen

Die folgende Tabelle beschreibt alle funktionalen Anforderungen. Jede Anforderung besitzt einen eindeutigen Bezeichner der Form FA-XXX.

Bezeichner	Funktion	Beschreibung
FA-010	Neues Spiel starten	Der Spieler kann über das Hauptmenü einen neuen Spielrun starten.
FA-020	Gebäude bauen	Der Spieler kann aus dem Buildmenü ein Gebäude auswählen und es auf einem freien Feld der Gridmap platzieren. Das Platzieren kostet Ressourcen.
FA-030	Gebäude abreißen	Der Spieler kann ein platziertes Gebäude abreißen.
FA-040	Bewohner zuweisen	Der Spieler kann Bewohner einem Gebäude zuweisen, um dessen Produktion zu aktivieren und Ressourcen zu generieren.
FA-050	Bewohner abziehen	Der Spieler kann Bewohner aus einem Gebäude entfernen.
FA-060	Spielgeschwindigkeit ändern	Der Spieler kann die Spielgeschwindigkeit anpassen.
FA-070	Spielstand laden	Der Spieler kann einen gespeicherten Spielstand laden.
FA-080	Einstellungen öffnen	Der Spieler kann auf das Einstellungsmenü zugreifen und dort Einstellungen zu beispielsweise der Musiklautstärke vornehmen.
FA-090	Spiel beenden	Der Spieler kann das Spiel über das Menü beenden.
FA-100	Ressourcen generieren	Das System generiert automatisch Ressourcen basierend auf den vorhandenen Gebäuden und zugewiesenen Bewohnern.
FA-110	Raid auslösen	Das System generiert in regelmäßigen Abständen Raids, die mit der Zeit stärker werden.
FA-120	Auf Game Over prüfen	Das System prüft kontinuierlich, ob die Game-Over-Bedingung erfüllt ist.
FA-130	Tagesablauf simulieren	Das System simuliert einen Tagesablauf, in dem Ressourcen produziert und verbraucht werden.
FA-140	Bewohner generieren	Das System generiert automatisch neue Bewohner basierend auf vorhandenen Wohnhäusern.
FA-150	Spielstand autosaven	Das System speichert den Spielstand automatisch in regelmäßigen Abständen.

5 Nicht-Funktionale Anforderungen

Die folgende Tabelle beschreibt die nicht-funktionalen Anforderungen. Jede Anforderung besitzt einen eindeutigen Bezeichner der Form NFA-XXX.

Bezeichner	Anforderung	Beschreibung
NFA-010	Plattformkompatibilität	Das Spiel muss auf Desktop-PCs unter Windows, macOS und Linux lauffähig sein.
NFA-020	Offline-Nutzbarkeit	Das Spiel muss vollständig offline nutzbar sein und darf keine Internetverbindung zum Spielen benötigen.
NFA-030	Leistungsanforderung	Das Spiel soll auf durchschnittlicher Hardware flüssig laufen; die grafischen und spielmechanischen Anforderungen sind deshalb auf einen ressourcenschonenden Desktop-Betrieb ausgelegt.
NFA-040	Benutzbarkeit	Die Bedienung soll für Gelegenheitsspieler verständlich und möglichst intuitiv sein.
NFA-050	Persistenz	Einstellungen und Spielstände müssen zuverlässig als JSON gespeichert und beim Laden validiert werden, sodass fehlende oder ungültige Werte durch Standardwerte ersetzt werden können.
NFA-060	Fehlertoleranz	Die Anwendung soll stabil mit Fehlern in Konfigurations- oder Savegame-Dateien umgehen und dabei keine unbenutzbaren Daten erzeugen.
NFA-070	Grafische Bedienung	Das Spiel soll ohne Konsolenbedienung nutzbar sein; die Steuerung erfolgt über die grafische Benutzeroberfläche.

6 Produktdaten

Ein Datensatz ist eine logisch zusammengehörige Menge gespeicherter Werte. Ein Datenobjekt ist die entsprechende Struktur im Code, die diese Werte zur Laufzeit hält und bei Bedarf als JSON speichert oder lädt.

6.1 Datensätze und Datenobjekte

`GameState` ist der zentrale Spielstand als Persistenz-Snapshot. Er enthält die Ressourcen, das Grid, den aktuellen Spieltag, geplante Raid-Termine und die gespeicherten Eventlog-Nachrichten.

`GameState.GridState` beschreibt das Spielfeld mit Breite, Höhe, allen Tiles und den platzierten Gebäuden.

`GameState.TileState` beschreibt ein einzelnes Grid-Feld mit Koordinaten, Tile-Typ, optionalem Gebäudetyp und optional zugewiesenen Dorfbewohnern.

`GameState.BuildingPlacementState` beschreibt eine Gebäudeplatzierung mit Gebäudetyp, Position und aktueller Arbeiteranzahl.

`GameSettings` speichert benutzerspezifische Einstellungen wie Fullscreen-Modus, Fenstergröße, Spielgeschwindigkeit sowie Lautstärke.

`ResolutionDto` beschreibt eine auswählbare Bildschirmauflösung mit Breite und Höhe.

`ResourceType` definiert die gespeicherten Ressourcentypen des Spiels, darunter Holz, Stein, Nahrung, Dorfbewohner, Kapazität und Soldaten.

Die Datei `settings.json` speichert die Benutzer- und Anzeigeeinstellungen, während `savegame.json` den gespeicherten Spielstand mit den aktuellen Spieldaten enthält.

7 Laufzeitumgebung [zum Betrieb der Software]

7.1 Software

Das Spiel wird auf Desktop-Betriebssystemen wie Windows, macOS oder Linux ausgeführt.

Als Laufzeitbasis dient eine Java-Umgebung beziehungsweise ein bereitgestelltes Desktop-
Runtime-Bundle.

Die grafische Laufzeit basiert auf libGDX und LWJGL3.

Für den Spielbetrieb ist keine Internetverbindung erforderlich.

7.2 Hardware

Das Spiel ist für Standard-Desktop-PCs und Laptops vorgesehen und kann mit Tastatur und Maus bedient werden.

Mittelklasse-Hardware soll ausreichen; das Spiel ist so ausgelegt, dass es auch auf niedriger Hardware flüssig läuft.

Die verwendete Grafikkarte muss die von LWJGL3 und libGDX benötigte Desktop-Grafik unterstützen.

7.3 (Orgware)

Das Spiel ist ausschließlich für den Einzelspielerbetrieb vorgesehen.

Der Start erfolgt über die bereitgestellte Desktop-Anwendung beziehungsweise über die ausführbare JAR-Datei.

Benutzerdateien werden im jeweiligen Profil des Betriebssystems gespeichert.

8 Entwicklungsumgebung [zur Entwicklung der Software]

8.1 Software

Als Entwicklungsumgebung wird IntelliJ IDEA verwendet.

Die Implementierung erfolgt in Java; der Quellcode ist auf Java-8-Kompatibilität ausgerichtet.

Für Build und Abhängigkeitsverwaltung wird Gradle mit Gradle Wrapper eingesetzt.

Das Spiel basiert auf dem Framework libGDX.

Git und GitHub dienen der Versionsverwaltung.

Checkstyle wird zur statischen Code-Prüfung verwendet, während Spotless den Quellcode formatiert.

GitHub Actions übernimmt automatisierte Prüfungen.

Typst wird für das Pflichtenheft verwendet, und Miro Board dient dem Brainstorming sowie der Planung.

8.2 Hardware

Für die Entwicklung wird ein Rechner oder Laptop mit ausreichend Leistung für IntelliJ IDEA, Gradle-Builds und das Starten des Spiels benötigt.

Maus und Tastatur sind für die Entwicklung vorgesehen.

Ein Internetzugang ist für das Abrufen von Abhängigkeiten, die Nutzung von GitHub und die Zusammenarbeit sinnvoll.

8.3 (Orgware)

Die Entwicklung erfolgt in Git-Banches.

Ein Zusammenführen auf main findet nur nach Code Review statt.

Die Teamabstimmung erfolgt über Miro, Aufgabenverteilung und Besprechungen.

9 Entwicklungshistorie

Die Entwicklung orientierte sich am SCRUM-Modell und erfolgte in Sprints. Die Sprintdauer wurde abhängig von den Terminen der anstehenden Übungsstunden des Moduls „Software-technik“ gewählt.

9.1 1. Sprint: 19.03.2026 - 30.03.2026

Feature	Autor	Notiz
Github Repository aufsetzen	Justin Hesse	https://github.com/justTschustin/lord-of-nothing
initiales libGDX-Projektsetup	Justin Hesse	
erste klickbare Grid-Logik	Justin Hesse	
checkstyle als Linter und spotless als Formatter hinzugefügt	Justin Hesse	Auch automatisiert in Github Pipeline

9.2 2. Sprint: 30.03.2026 - 09.04.2026

Feature	Autor	Notiz
Spritedesign für House	Luca Kaden	
Placeholder Sprites für Gebäude	Luca Kaden	sowohl für Gebäude, die 1x1 Tiles einnehmen, als auch 1x2 und 2x2
Sidebar	Valentin Marcinek	Sidebar neben dem Spielgrid, wo Gebäude zum Platzieren ausgewählt werden können
House als erstes platzierbares Gebäude	Valentin Marcinek	mit finalem Sprite
Holz als erste Ressource integriert	Valentin Marcinek	Beinhaltet auch Anpassung, dass Ressourcen verbraucht werden, wenn House platziert wird
Sawmill (Sägewerk) platzierbar	Valentin Marcinek	mit Placeholder Sprite
Quarry (Steinbruch) platzierbar	Valentin Marcinek	mit Placeholder Sprite
Field (Feld) platzierbar	Valentin Marcinek	mit Placeholder Sprite
Gebäudeinspektor	Valentin Marcinek	sichtbar bei anklicken eines platzierten Gebäudes auf der rechten Seite
Hauptmenü	Justin Hesse	minimale Funktionalität (Spiel starten, Einstellungen, Spiel beenden)
Einstellungsmenü	Justin Hesse	generelle Logik mit Vollbild/Fensteranwendung-Switch, Auflösungsselector
Pausenmenü	Justin Hesse	Button oben rechts im Spiel, welches das Spiel pausiert; ermöglicht

		auch öffnen des Einstellungsme- nüs
--	--	--

9.3 3. Sprint: 09.04.2026 - 16.04.2026

Feature	Autor	Notiz
Spielgrid überarbeiten	Christopher Aaron Sabbach	Gridtiles neu skaliert, Gridgröße mit Sidebar garantieren
Ticksystem	Justin Hesse	Zeitberechnung im Spiel -> zeigt Tag und Stunde an
anpassbare Spielgeschwindigkeit	Justin Hesse	verändert Tempo des Tick-systems
Dorfbewohner kommen ins Dorf	Justin Hesse	kommen im Spiel jeden Tag (Ticksystem) an
Speichern der Einstellungen	Justin Hesse	Einstellungen werden als JSON gespeichert und beim Starten wieder geladen
Sawmill Sprite	Luca Kaden	
Sawmill platzierbar	Valentin Marcinek	mit finalem Sawmill Sprite

9.4 4. Sprint: 16.04.2026 - 23.04.2026

Feature	Autor	Notiz
Gebäude entfernen	Christopher Aaron Sabbach	geben Spieler 50% der Baukosten zurück
Gebäude Preview beim Platzieren	Christopher Aaron Sabbach	
Placeholder Sprites für Ressourcen	Luca Kaden	Holz, Nahrung, Dorfbewohner, Steine, Soldaten, Zeit
Gebäude-/Ressourcengenerierungslogik	Valentin Marcinek	Dorfbewohner können Gebäude zugeordnet werden und Gebäude generiert Ressourcen, wenn Bewohner diesem zugeordnet sind
Kaserne	Valentin Marcinek	platzierbar, zugeordnete Bewohner zählen als Soldaten

9.5 5. Sprint: 23.04.2026 - 05.05.2026

Feature	Autor	Notiz
Hauptmenü Hintergrund	Christopher Aaron Sabbach	
Gebäudesprites	Luca Kaden	Sprites für Kaserne, Quarry, Feld
Spiel speichern und laden	Justin Hesse	als JSON gespeichert

Popups	Justin Hesse	Popups im Spiel mit Nachrichten an den Spieler, welche das Spiel pausieren und weggeklickt werden
minimale Raid-Logik	Justin Hesse	In definierten Intervallen werden Banditenüberfälle via Popup angekündigt und geschehen via Popup (wie viele angreifen, ohne Gameover, ohne Verluste)
Pflichtenheft, Dokumentation	alle	WIP, im Fokus

9.6 6. Sprint: 05.05.2026 - 11.05.2026

Feature	Autor	Notiz
Sprites für Ressourcen	Luca Kaden	Sprites für Holz, Stein, Nahrung, Bewohner, Zeit, Soldaten
Hintergrundmusik	Luca Kaden	verstellbare Lautstärke in Einstellungen
Raid-Logik	Justin Hesse	Verluste anhand einer Gaußschen Glockenkurve berechnet (je nach Anzahl der Angreifenden und Verteidigenden), GameOver
GameOver-Screen	Justin Hesse	durch Raid möglich
Eventlog	Valentin Marcinek	Eventlog in unterer rechter Ecke, Log für Popup-Nachrichten, Ankünfte von Bewohnern
Menü-/HUD-Texturen	Valentin Marcinek	
Sounds	Valentin Marcinek	Menü-Klick-Sound, Gebäude-Platzieren-Sound
Raid Banner Texturen	Christopher Aaron Sabbach	Design für Raid Banners (Ankündigungen, Geschehen)
Interface/Menü Design Overhaul	Christopher Aaron Sabbach	Implementierung neuer HUD-Texturen, Hauptmenü Hintergrund
Raid Design Overhaul	Christopher Aaron Sabbach	Implementierung neuer Raid Banner mit Soundeffekten

9.7 7. Sprint: 11.05.2026 - 17.05.2026

Feature	Autor	Notiz
Playtesting & Balancing	alle	Intervalle, Generierung, Angreifen usw. anpassen

Dokumentation	alle	
---------------	------	--